

CARDS BATTLE

Álvaro García Zafra

DOCUMENTACIÓN

Objetivo

El objetivo a lograr en esta práctica era poder conseguir que cada marcador tenga una tropa asociada, tener la aplicación aplicada en directo, mostrando la vida de cada tropa, a quién le corresponde atacar y quién ha ganado.

Por otro lado se pretendía enlazar esto con el uso de reconocimiento de audio, el cuál lo he aplicado para asociar tropas a marcadores y para el transcurso de la batalla, y con el reconocimiento facial, aplicado para llevar a cabo la conciencia de contexto, permitiendo en función del usuario que cargue directamente su favorita.

Esquema de ficheros

- [prCUIA.py](#): archivo ejecutado para activar la aplicación, contiene todo lo relativo al uso de los siguientes ficheros y también incorpora la captación y el procesamiento del audio.
 - [cuia.py](#) archivo para trabajar con marcadores e impresión de modelos
 - `estado_de_juego.py`: archivo encargado de todas las funciones de actualización de estado
 - `reconocimiento_facial.py`: archivo encargado de todas las funciones de procesamiento de captura y procesamiento de usuarios a través de su cara, para esto utilizo un archivo json y guardo las imágenes en una carpeta.
 - imágenes: carpeta con las distintas tropas

Análisis del código

En el análisis del código nos vamos a centrar en el funcionamiento del fichero principal y algunas funciones del archivo reconocimiento_facial, respecto al estado de la partida no voy a incluir ninguna captura de como se va actualizando debido a que no es el objetivo principal de la práctica.

En primer lugar vamos a hablar de la llamada a identificar o registrar usuario, la cuál se hace en nuestro archivo principal:

```
23 usuario = reconocimiento_facial.identificar_o_registrar()
```

```
def identificar_o_registrar():
    usuarios = cargar_usuarios()
    recognizer = cargar_modelo()

    if recognizer is None:
        print("No hay modelo entrenado. Se necesita registrar al primer usuario.")
        nombre = capturar_nombre_por_voz()
        if nombre:
            return registrar_usuario(nombre)
        return None

    cam = cv2.VideoCapture(0)
    detector = cv2.CascadeClassifier(DETECTOR_PATH)
    identificado = False
    usuario = None

    print("Escaneando rostro...")
    tiempo_inicio = time.time()
    tiempo_limite = 10 # segundos

    while True:
        ret, frame = cam.read()
        if not ret:
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        rostros = detector.detectMultiScale(gray, 1.3, 5)

        for (x, y, w, h) in rostros:
            rostro = gray[y:y+h, x:x+w]
            id_predicho, conf = recognizer.predict(rostro)

            if conf < 70:
                usuario = next((u for u in usuarios if u["id"] == id_predicho), None)
                if usuario:
                    identificado = True
                    cv2.putText(frame, f"usuario['nombre']", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)
                    break

            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

```

cv2.imshow("Reconocimiento Facial", frame)

if identificado:
    break

# Si el tiempo se agota sin identificación, salir del bucle para registrar
if time.time() - tiempo_inicio > tiempo_limite:
    print("Tiempo de reconocimiento agotado.")
    break

if cv2.waitKey(1) & 0xFF == 27: # Opción de salida manual opcional
    break

cam.release()
cv2.destroyAllWindows()

if identificado:
    print(f"¡Hola {usuario['nombre']}! Bienvenido.")
    return usuario
else:
    print("No se reconoció el rostro. Registrando nuevo usuario...")
    nombre = capturar_nombre_por_voz()
    if nombre:
        return registrar_usuario(nombre)
    return None

```

En dicha función podemos ver que en primer lugar detecta si ya hay usuarios, si no los hay o no reconoce el rostro a los 10 segundos pide el nombre del usuario, le escanea el rostro y lo identifica, por otro lado si el usuario existe le escanea la cara e inicia la aplicación.

```

# mientras que no haya 4 tropas seleccionadas
if not salir:
    # Cargar favoritas por comando
    if "cargar lista de favoritos" in texto.lower():
        if usuario and "favoritas" in usuario:
            for i, carta in enumerate(usuario["favoritas"]):
                if i >= 4:
                    break
                if carta == "dragon_fuego":
                    imagenes[i] = cv2.cvtColor(cv2.imread("imagenes/dragon_fuego.jpg"), cv2.COLOR_BGR2BGRA)
                    estado_de_juego.crear_tropa(i, 0, tropas)
                elif carta == "dragon_agua":
                    imagenes[i] = cv2.cvtColor(cv2.imread("imagenes/dragon_agua.jpg"), cv2.COLOR_BGR2BGRA)
                    estado_de_juego.crear_tropa(i, 1, tropas)
                elif carta == "golem":
                    imagenes[i] = cv2.cvtColor(cv2.imread("imagenes/golem.png"), cv2.COLOR_BGR2BGRA)
                    estado_de_juego.crear_tropa(i, 2, tropas)
                elif carta == "erizo":
                    imagenes[i] = cv2.cvtColor(cv2.imread("imagenes/erizo.png"), cv2.COLOR_BGR2BGRA)
                    estado_de_juego.crear_tropa(i, 3, tropas)
            marcadores_seleccionados = len(usuario["favoritas"])
            if marcadores_seleccionados >= 4:
                salir = True
            print("Favoritas cargadas.")
        else:
            print("No hay favoritas guardadas para este usuario.")
        return

    # Modificar lista de favoritas por voz
    elif "modificar lista de favoritos" in texto.lower():
        nuevas_favoritas = []
        print("Dime las cartas que quieres guardar como favoritas (máximo 4). Di 'fin' para terminar antes de tiempo.")
        with sr.Microphone() as source:
            while len(nuevas_favoritas) < 4:
                try:
                    print("Escuchando carta...")
                    audio = recognizer.listen(source, timeout=5)
                    carta = recognizer.recognize_google(audio, language="es-ES").lower()
                    print(f"Detectado: {carta}")

                    if "fin" in carta:
                        break
                    elif "dragón de fuego" in carta:
                        nuevas_favoritas.append("dragon_fuego")
                    elif "dragón de agua" in carta:
                        nuevas_favoritas.append("dragon_agua")
                    elif "golem" in carta:
                        nuevas_favoritas.append("golem")
                    elif "erizo" in carta:
                        nuevas_favoritas.append("erizo")
                    else:
                        print("Carta no reconocida.")
                except sr.UnknownValueError:
                    print("No entendí. Repite por favor.")
                except sr.WaitTimeoutError:
                    print("No se detectó voz. Intenta de nuevo.")

```

Una vez ha detectado el rostro le permite asociar a cada marcador una tropa en función de su baraja favorita guardada, para lo que solo tendría que decir cargar lista de favoritos, también le da la opción de editar esta si todavía no la ha cargado en la partida, para editarla simplemente se dice modificar lista de favoritos y se van indicando las tropas que quieres añadir, no es necesario que se asocien a favoritos una tropa para cada marcador.

```

elif "añadir erizo" in texto.lower():
    if marcadores_seleccionados < 4:
        imagenes[marcadores_seleccionados] = cv2.cvtColor(cv2.imread("imagenes/erizo.png"), cv2.COLOR_BGR2BGRA)
        estado_de_juego.crear_tropa(marcadores_seleccionados, 3, tropas)
        marcadores_seleccionados += 1
        print("Se añadió un Erizo.")
    else:
        print("Ya hay 4 tropas.")
else:
    print("Comando no válido o no permitido en esta fase.")

print(f"Marcadores seleccionados: {marcadores_seleccionados}/4")
if marcadores_seleccionados >= 4:
    salir = True
    print("Tropas completas. Comienza la batalla.")
return

```

Por otro lado si no queremos cargar la lista de favoritos podemos ir añadiendo una a una las tropas, para lo cuál habría que decir añadir “nombre tropa”, en el momento que estén todas añadidas dará comienzo la partida.

Transcurso de la partida:

```
# Proceso de los combates
# cambio de tropa
if "cambiar tropa delantera" in texto.lower():
    if id_jugador == 1:
        tropa_antes_cambio = tropa_delantera1
        tropa_delantera1 = estado_de_juego.cambiar_tropa_delantera(tropa_delantera1, id_jugador, tropas)
        if tropa_delantera1 is None: # ambas tropas muertas
            print("¡No te quedan tropas para cambiar!")
        elif tropa_antes_cambio != tropa_delantera1: # Cambio de tropa delantera
            print(f"Jugador 1 cambió la tropa delantera a {tropa_delantera1}.")
            id_jugador = 2
        else:
            print("No se pudo cambiar la tropa delantera (la otra tropa podría estar sin vida o ya es la delantera).")
    else:
        tropa_antes_cambio = tropa_delantera2
        tropa_delantera2 = estado_de_juego.cambiar_tropa_delantera(tropa_delantera2, id_jugador, tropas)
        if tropa_delantera2 is None: # Both troops dead
            print("¡No te quedan tropas para cambiar!")
        elif tropa_antes_cambio != tropa_delantera2: # Successful swap
            print(f"Jugador 2 cambió la tropa delantera a {tropa_delantera2}.")
            id_jugador = 1
        else:
            print("No se pudo cambiar la tropa delantera (la otra tropa podría estar sin vida o ya es la delantera).")
elif "atacar con la primera tropa" in texto.lower():
    if id_jugador == 1:
        if estado_de_juego.tropa_con_vida(tropa_delantera1, tropas):
            print(f"Jugador 1 ataca con su tropa delantera (ID: {tropa_delantera1}).")
            estado_de_juego.atacar(tropa_delantera1, tropa_delantera1, tropa_delantera2, id_jugador, tropas)
            id_jugador = 2
        else:
            print("No se puede atacar con la tropa delantera porque no tiene vida.")
    else:
        if estado_de_juego.tropa_con_vida(tropa_delantera2, tropas):
            print(f"Jugador 2 ataca con su tropa delantera (ID: {tropa_delantera2}).")
            estado_de_juego.atacar(tropa_delantera2, tropa_delantera2, tropa_delantera1, id_jugador, tropas)
            id_jugador = 1
        else:
            print("No se puede atacar con la tropa delantera porque no tiene vida.")
elif "atacar con la segunda tropa" in texto.lower():
    if id_jugador == 1:
        if estado_de_juego.tropa_con_vida(tropa_trasera1, tropas):
            print(f"Jugador 1 ataca con su tropa trasera (ID: {tropa_trasera1}).")
            estado_de_juego.atacar(tropa_trasera1, tropa_delantera1, tropa_delantera2, id_jugador, tropas)
            id_jugador = 2
        else:
            print("No se puede atacar con la tropa trasera porque no tiene vida.")
    else:
        if estado_de_juego.tropa_con_vida(tropa_trasera2, tropas):
            print(f"Jugador 2 ataca con su tropa trasera (ID: {tropa_trasera2}).")
            estado_de_juego.atacar(tropa_trasera2, tropa_delantera2, tropa_delantera1, id_jugador, tropas)
            id_jugador = 1
        else:
            print("No se puede atacar con la tropa trasera porque no tiene vida.")
elif "me rindo" in texto.lower():
    if id_jugador == 1:
        perdedor[0] = True
        print("El Jugador 1 se ha rendido. ¡El Jugador 2 gana!")
    else:
        perdedor[1] = True
        print("El Jugador 2 se ha rendido. ¡El Jugador 1 gana!")
    sys.exit()
else:
    print("Comando no reconocido. Intenta 'atacar con la primera tropa', 'atacar con la segunda tropa', 'cambiar tropa delantera', o 'me rindo'.")
```

Como podemos ver a la hora de realizar las batallas tenemos una serie de comandos de voz, nos podemos encontrar con cambiar tropa, atacar con la primera y segunda tropa y rendirse. En todo momento hay comprobaciones y actualizaciones del estado de la partida, prohibiendo acciones que no deben ocurrir, como que un jugador muerto no puede hacer daño.

```

# Escucha en segundo plano
r = sr.Recognizer()
m = sr.Microphone()
with m as source:
    r.adjust_for_ambient_noise(source)
    print("Micrófono calibrado. Escuchando...")
stop_listening = r.listen_in_background(m, voz_callback)
💡
diccionario = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_5X5_50)
detector = cv2.aruco.ArucoDetector(diccionario)

cam = 0
bk = cuia.bestBackend(cam)
webcam = cv2.VideoCapture(cam, bk)
ancho = int(webcam.get(cv2.CAP_PROP_FRAME_WIDTH))
alto = int(webcam.get(cv2.CAP_PROP_FRAME_HEIGHT))
webcam.release()

```

variables para la escucha en segundo plano, cargar los marcadores aruco y conectar correctamente la cámara.

```

try:
    import camara
    cameraMatrix = camara.cameraMatrix
    distCoeffs = camara.distCoeffs
except ImportError:
    cameraMatrix = np.array([
        [1000, 0, ancho / 2],
        [0, 1000, alto / 2],
        [0, 0, 1]
    ])
    distCoeffs = np.zeros((5, 1))

ar = cuia.myVideo(cam, bk)

def origen(TAM):
    return(np.array([[-TAM/2.0, -TAM/2.0, 0.0],
                    [-TAM/2.0, TAM/2.0, 0.0],
                    [TAM/2.0, TAM/2.0, 0.0],
                    [TAM/2.0, -TAM/2.0, 0.0]]))

def proyeccion(puntos, rvec, tvec, cameraMatrix, distCoeffs):
    if isinstance(puntos, list):
        return(proyeccion(np.array(puntos, dtype=np.float32), rvec, tvec, cameraMatrix, distCoeffs))
    if isinstance(puntos, np.ndarray):
        if puntos.ndim == 1 and puntos.size == 3:
            res, _ = cv2.projectPoints(puntos.astype(np.float32), rvec, tvec, cameraMatrix, distCoeffs)
            return(res[0][0].astype(int))
        if puntos.ndim > 1:
            aux = proyeccion(puntos[0], rvec, tvec, cameraMatrix, distCoeffs)
            aux = np.expand_dims(aux, axis=0)
            for p in puntos[1:]:
                aux = np.append(aux, [proyeccion(p, rvec, tvec, cameraMatrix, distCoeffs)], axis=0)
            return(np.array(aux))

```

El código que estamos viendo servirá para que funcione el apartado de realidad aumentada, calibrando la cámara definiendo los puntos en el espacio y proyectando imágenes en 2D.

```

def variasimagenes(frame):
    framebgra = cv2.cvtColor(frame, cv2.COLOR_BGR2BGRA)
    hframe, wframe, _ = frame.shape
    tam = 0.1
    bboxes, ids, _ = detector.detectMarkers(frame)
    if ids is not None:
        res = framebgra
        for i in range(len(ids)):
            ret, rvec, tvec = cv2.solvePnP(origen(tam), bboxes[i], cameraMatrix, distCoeffs)
            if ret:
                id_marcador = ids[i][0]

                if id_marcador < len(imagenes):
                    imagen = imagenes[id_marcador]
                    tropa_vida = tropas[id_marcador][0]

                    # Convertir a gris si está muerta
                    if tropa_vida <= 0:
                        gris = cv2.cvtColor(imagen[:, :, :3], cv2.COLOR_BGR2GRAY)
                        gris_color = cv2.cvtColor(gris, cv2.COLOR_GRAY2BGRA)
                        gris_color[:, :, 3] = imagen[:, :, 3]
                        imagen = gris_color

                    # Proyección de imagen al marcador
                    h, w, _ = imagen.shape
                    desde = np.array([[0, 0], [w, 0], [w, h], [0, h]])
                    hasta = proyeccion(origen(tam), rvec, tvec, cameraMatrix, distCoeffs)
                    M = cv2.getPerspectiveTransform(np.float32(desde), np.float32(hasta))
                    warp = cv2.warpPerspective(imagen, M, dsize=(wframe, hframe))
                    res = cuia.alphaBlending(warp, res)

                    # ----- TEXTO FLOTANTE 3D -----
                    posicion_3D_texto = np.array([[0, 0, -0.05]])
                    punto_2D, _ = cv2.projectPoints(posicion_3D_texto, rvec, tvec, cameraMatrix, distCoeffs)
                    x, y = punto_2D[0][0].astype(int)

                    if tropa_vida > 0:
                        texto = f"{int(tropa_vida)} HP"
                        color_texto = (255, 255, 0, 255)
                    else:
                        texto = "MUERTA"
                        color_texto = (0, 0, 255, 255)

                    # Mostrar texto
                    cv2.putText(res, texto, (x - 30, y), cv2.FONT_HERSHEY_SIMPLEX,
                                0.8, color_texto, 2, cv2.LINE_AA)
            else:
                res = frame

    # Mostrar vida como texto en la esquina superior izquierda
    if perdedor[0]:
        cv2.putText(res, f"Ha ganado el jugador : 2", (10, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 128, 128), 2)
    elif perdedor[1]:
        cv2.putText(res, f"Ha ganado el jugador : 1", (10, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 128, 128), 2)
    elif perdedor[0] and perdedor[1]:
        cv2.putText(res, f"Ha habido un empate", (10, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 128, 128), 2)
    else:
        cv2.putText(res, f"Jugador 1 tropa al frente: {tropa_delantera1}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 128, 128), 2)
        cv2.putText(res, f"Jugador 2 tropa al frente: {tropa_delantera2}", (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 128, 128), 2)
        cv2.putText(res, f"Turno: {id_jugador}", (10, 90), cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 128, 128), 2)

    return res

```

Esta función detecta los marcadores y proyecta las imágenes asociadas a cada uno de ellos, además en cada uno de ellos muestra su vida restante. También muestra en la pantalla alguna información del estado, como quien es el ganador, de quién es el turno y que tropas están al frente.

```
ar.process = variasimagenes

try:
    ar.play("CUIA Álvaro García", key=ord(' '))
    while not exit_flag:
        continue
finally:
    stop_listening(wait_for_stop=False)
    ar.release()
```

Por último nos encontramos con la parte que se encarga de iniciar el sistema de video de la aplicación, el while se encargará de mantener la aplicación activa mientras se reproduce la voz. Una vez uno de los jugadores ha ganado bastará con tocar la tecla espacio para cerrar la aplicación.